

**Full Stack Development / Further Web Programming COSC2758 UG  
/ COSC2938 PG (Semester 1, 2026)  
Assignment 2**

|                         |                                                                                                                                                                                                                                                                                                                  |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Assessment Type</b>  | To be attempted in a <u>group of 2</u> .<br><br><b>You will need to register your A2 group on Canvas and on GitHub Classroom by Friday Week 9.</b> A request to working solo must have a strong reason, and the permit to do that must be obtained via email by Friday Week 9.                                   |
|                         | Submit online via Canvas → Assignments → Assignment 2.<br><br>Marks awarded for meeting requirements as closely as possible. Please read the rubrics for details. Clarifications/updates may be made via announcements/relevant discussion forums.                                                               |
| <b>Due Date</b>         | Week 13, Sunday 7 <sup>th</sup> June 2026, 11:59 pm <i>Melbourne Time</i><br><br><b>This is a longer assessment, start in time to avoid delays and unexpected last-minute issues.</b>                                                                                                                            |
| <b>Marks</b>            | 45                                                                                                                                                                                                                                                                                                               |
| <b>Document control</b> | Published on 24/04/2026. Please consider this as a draft, and review and provide feedback about this document if you see any typo, ambiguity, conflicting info, or need clarification by 30/04/26. After that this document should be considered final.<br><br>Removed ambiguity for data visualization on 21/05 |

## 2.1 Overview & Learning outcomes

In this assignment, you will develop a Full Stack Web Application to complete the front-end prototype built from assignment 1. You are required to use the following stacks:

- **Frontend:** React TS
- **Backend :** Node & Express with TypeORM
- **Relational Database:** Cloud MS SQL Server
- **Server deployment:** Render
- Using stacks other than the ones mentioned above will **FETCH A ZERO** for the whole assignment.
- Using object-oriented react components will **result in a ZERO** for the whole assignment.

The tasks are divided into *four* parts: PA (Pass), CR (Credit), DI (Distinction) & HD (High Distinction).

The **HD** section tasks will require self-research; you will not get straight answers in the course material. While we are happy to assist you with those tasks, you must do most of your work and research. This is done on purpose to prepare you for future work and the rigours of the IT industry.

If you find a specification open to interpretation, post a query identifying the specification in the corresponding discussion board for assignment 2. Software development in real life does not come with a definitive roadmap and flowcharts complete with instructions. Often, it is the job of the developer to clarify client requirements. For this assignment and course, the lecturer is considered as the client, and the tutor is considered as Product Owner who represents the client.

You are responsible for regularly attending lectorial, practical and consultation session(s). Tips:

- Bring your questions to online discussion board and consultation sessions.
- Watch the online lectorial recordings regularly if you cannot attend the face-to-face session.
- Do NOT start the work on assignment at the last minute.
- Do NOT ask for last-minute extensions, these are often rejected. Extensions can only be granted for personal and medical reasons, provided you can supply some documentary evidence.
- If you have any issues regarding teammates, please raise them as early as possible. Team changes will not be accepted after Week 10.

This assessment relates to the following learning outcomes of the course which are:

CLO2: Demonstrate proficiency with a web application development framework

CLO3: Implement a range of techniques and procedures for developing a small to medium-scale web

application CLO4: Demonstrate knowledge of and utilise software engineering patterns in development

CLO5: Design and manage the development life cycle of a complete application.

## 2.2 GitHub & Submission

You will be marked on the use of GitHub and the development process. In week 1-2 you were registered on **rmit-fsd-2026-s1** GitHub organisation. As a member of this, you will be able to register your A2 group on GitHub Classroom. Please read the following:

- **DO NOT work on Assessment 1 repository as these assessments are being marked.** Copy the code from **Assessment 1 repository to a new Assessment 2 repository.**
- Since you are working in a group, please make sure your team's name on GitHub Classroom is the same as your team's name on Canvas.
- Please find your student ID in the list and put yourself in the correct A2 group or create the A2 group, do not join the old A1 group: <https://classroom.github.com/assignment-invitations/cdbbbecf3003b56e9771f119f67e310e>
- Once registered, a team repo will be made available. Please take note of the address of this repo and include the URL of your GitHub repository in the README file.



You need to submit **one zipped archive** containing:

- *your whole project folder (frontend, backend, admin-frontend, admin-backend) **without** node\_modules directory*
- *README file containing GitHub URL, production server web services URLs (HD section)*
- *Group Contribution Form (pdf)*
- *Entity Relationship Diagram showing your implemented DB tables (pdf)*
- *User Story list (xls)*

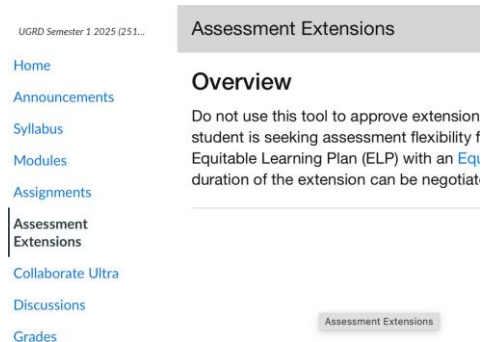
Assessment declaration: When you submit work electronically, you agree to the assessment

declaration: <https://www.rmit.edu.au/students/my-course/assessment-results/assessment>

## 2.3 Extension requests

After the due date, you will have **five** business days to submit your assignment as a late submission. Late submissions will incur a penalty of 10% per day. After these 5 days, Canvas will be closed, and you will lose ALL the assignment marks.

All extension requests must be made via the **Assessment Extensions** page of the Course canvas (*screenshot below*):



Because your extension request outcome impacts your teammate, please make sure you have their consent when you apply for extension and ask them to request for the same due date using the same reason as yours. An approval given to one of the team members does not automatically warrant approval to their teammate, so both team members need to submit and get approval.

## 2.4 Academic integrity and plagiarism

Academic integrity is about the honest presentation of your academic work. It means acknowledging the work of others while developing your insights, knowledge and ideas. You should take extreme care that you have:

- acknowledged words, data, diagrams, models, frameworks and/or ideas of others you have quoted (i.e. directly copied), summarised, paraphrased, discussed or mentioned in your assessment through the appropriate referencing methods,
- provided a reference list of the online code used so your reader can locate the source if necessary. This includes material taken from Internet sites **& your own course material**.

If you fail to cite the sources of your material properly, you risk being accused of plagiarism for presenting someone else's work and ideas as your own without proper attribution.

RMIT University treats plagiarism as a very serious offence constituting misconduct. Plagiarism covers a variety of inappropriate behaviours, including:

- Contract cheating- paying someone to do your work
- Failure to properly document a source
- Copyright material from the internet or databases
- Collusion between students
- Posting assignment tasks on technical forums (*reddit, stack exchange, etc.*) and asking for solution(s)
- Copying code with the aid of generative AI tools.

For further information on our policies and procedures, please refer to:

<https://www.rmit.edu.au/students/student-essentials/assessment-and-results/academic-integrity>

## 2.5 Use of generative AI tools

As we increasingly integrate technology into our academic and professional practices, it is imperative to address the use of generative AI tools, such as ChatGPT, in your university assessments. These tools should be used cautiously, as they function by synthesising and repackaging existing text from various sources. We will discuss the implications of using such technologies, including the academic standards and their significance in your future careers, during lectorial sessions. We encourage you to produce and submit only your original work and to seek clarification from instructors if you have any uncertainties.

Furthermore, please note that the Co-pilot feature will **not** be enabled within the official RMIT's Git organisation for this course (**rmit-fsd-2026-s1**). If you choose to use Co-pilot independently, it is crucial to use it responsibly. Remember, generative AI should not be used to replicate code directly.



We are committed to academic integrity and will conduct face-to-face meetings to address any cases of suspicious code submissions.

Ensure you properly reference any code that AI tools generate to maintain transparency and uphold academic honesty. Here are RMIT's official AI-related referencing guidelines:

[https://rmit.libguides.com/referencing\\_AI\\_tools#s-lg-box-22377324](https://rmit.libguides.com/referencing_AI_tools#s-lg-box-22377324)

## 2.6 Assessment scenario overview & system requirements

The software company that produces the Venue Vendor application has accepted the **VV** front-end prototype and now recommends that a full-stack version of a website be developed. The company recommends including extra features and constraints for the website- a summary is presented here; you will find more details in **Section 2.7: Tasks**.

### System requirements:

- 1) The company wants the developer to use a Cloud MS SQL Server database (**Server: dipto-database.cn2ems8y2mfe.ap-southeast-2.rds.amazonaws.com; Database: a team member's database**) for backend purposes. You are encouraged to use VS Code DB extension like mssql or a standalone DBMS client like DBeaver to administer your database tables.
- 2) The TypeScript code must be compiled without errors when executing the `npm run build` command to get full marks for a requirement.
- 3) The full-stack version of VV must be a multi-user website that supports **three** types of users: hirer, vendor, and admin.
- 4) The hirer can make a booking request for a timeslot for an event / occasion at a venue managed by a vendor.
- 5) A vendor manage at least one venue. A hirer can book multiple timeslots at any venue.
- 6) A vendor can only see their venues in the system. A hirer can see all venues by all vendors. If you are attempting only the PA, CR & DI sections, manually assign venues to vendors, in the backend database.
- 7) The VV admin dashboard must be on a separate website. This will be mentioned in the HD section of the assignment.
- 8) The website **must** be fully styled and look professional. The content must make sense i.e., use of lorem ipsum is **not** allowed.
- 9) Only well-established and documented third-party libraries/modules **should** be used. If you are in doubt about a module's legitimacy, contact your lecturer for advice.
- 10) The digital assets (images, icons, audio & video) must be outsourced from royalty-free websites. You should **not** steal someone else's assets to enhance the look and feel of your website. High-quality & free assets can be obtained from:
  - <https://unsplash.com/> (images)
  - <https://uifaces.co/> (avatar)
  - <https://fonts.google.com/icons?selected=Material+Icons:home> (icons)

*The assessment tasks are outlined in the next section➡*

## 2.7 Tasks (please ensure you have read the 10 requirements in section 2.6)

To proceed to higher parts, **you must complete** all the specifications in the lower part; you must **not** cherry-pick specifications from various parts. For example, complete all the specifications in the PA part before proceeding to the CR part, and so on. **Use of object-oriented React / React JS /technology stacks other than what is mentioned on page 1 will FETCH A ZERO for the whole assignment.**

Here is the architecture diagram for the website:

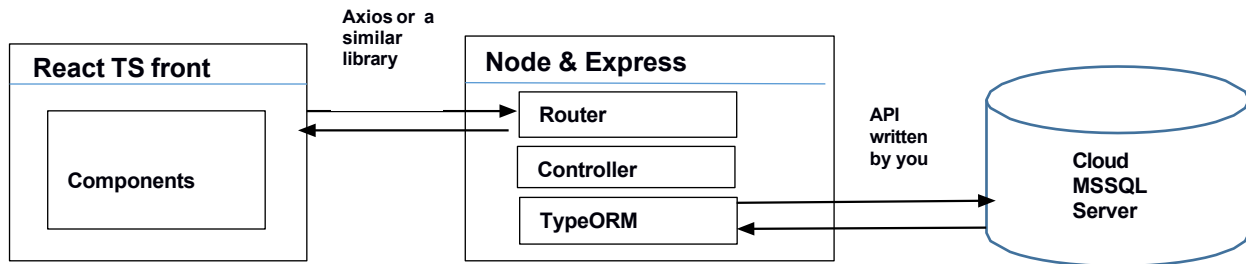


Fig. 1 Full Stack Architecture Diagram

React front-end app talks to API defined in Node + Express layer via Axios library. REST API created by you in Node + Express layer (*backend*) communicates with the database. Create separate projects – one for React frontend app & one for Node + Express + TypeORM layer.

### PA part [23 marks]

#### a. (3 marks) Database schema model files

Create an ER (Entity-Relationship) diagram that will represent the database schema for the website. It should display the tables with the fields, keys, constraints, and relationships between the tables. **Think of these questions:** *How many tables do I need? Which fields do I need in these tables? What data types should these fields use? What kind of relationships exist among these tables? Is the database normalised? (i.e., avoid duplicated data, do not use too few or too many tables).*

Place this model diagram inside the zip file. You may use a scanned image or a PDF document.

Using TypeORM in the Node & Express (backend) project, create model files that represent the above tables, keys, and constraints.

#### b. (3 marks) User Story list.

This assignment specification contains core and broad requirements of the web applications. You need to expand the details and come up with your own User Story list to work out the workflow, overall User Interface, User Experience and acceptance criteria for each feature. Please work closely with your Product Owner to confirm your understanding of the requirements, and produce a spreadsheet that holds the following columns:

- Epic (what is the theme, what group of problems are you solving)
- Persona (who is the user, who has the problems that you are trying to help solve using your website / app)

- User Story (a specific task requirement that can be done on and with the system)

In the following format: “As a \_\_\_\_ (persona) I want to (perform a task) so that \_\_\_\_ (achievement / goal)”

- Acceptance Criteria (how you prove that your feature worked and the system behaves as expected)

In the following format: “

Given \_\_\_\_ (the context, pre-conditions, on what web / application page or exact section of a page)

When \_\_\_\_ (an action / trigger)

Then \_\_\_\_ (system behaviour / system response as consequence / outcome)”

The user story list is a live document that you will keep updating throughout the project. You may use and continue using the existing User Story list that your tutor has helped you with in A1 as starting point and expand it to include the extra columns. You need to submit the final version of it in the zip file.

c. (5 marks) Sign-up & Sign-in

Implement the sign-up and sign-in features from assignment 1. This time the user details are stored in the MS SQL Server database. The API in the backend should handle all the database operations. All the form input validations must be handled on the React end, as well as being validated in the backend / REST API.

Upon successful log in, the user must see a welcome message in the format of *Welcome* username. Introduce a logout link for a logged in user.

Add a new **profile feature**- this component shows a single hirer's information in the main content area. The completed Profile will display user details, and the date of joining.

Add a complex password validator so that each password should contain at least 1 upper case letter, 1 lower case letter, and 1 special character, and be at least 6 digits.

**ATTENTION postgraduate students (COSC2938):** candidate should be able to choose/edit an avatar on the profile page. *How and from where will you source this avatar? Where will you store it? Do you need to store it?*

d. (6 marks) Hirer's page

Implement all the requirements for the hirer's page from Assessment 1 with the following changes:

CHANGE 1: all the relevant details are stored in the MS SQL Server database. The API in the backend should handle all the database operations. All the form input validations must be handled on the React end, as well as being validated in the backend / REST API.

e. (6 marks) Vendor's page

Implement all the requirements for vendor's page from assessment 1 with the following changes:

CHANGE 1: all the relevant details are stored in the MS SQL Server database. The API in the backend should handle all the database operations. All the form input validations must be handled on the React end, as well as being validated in the backend / REST API.

**CR [8 marks]**

f. Implement the additional features for hirers and vendors from assessment 1 with the following changes:

CHANGE 1: all the relevant details are stored in the MS SQL Server database. The API in the backend should handle all the database operations. All the form input validations must be handled on the React end, as well as being validated in the backend / REST API.

CHANGE 2: Hirer should be able to check a venue's 'recommended suitability' based on pre-defined keywords, e.g. tennis, dinner, classical music, rock concert, birthday, wedding.

CHANGE 3: Vendor should be able to perform CRUD operation on their own venue(s).

CHANGE 4: Vendor should be able to block / unblock any timeslot (make a timeslot unavailable for booking).

CHANGE 5: Vendor should be able to see a booking list for all venues under their management and view hirer's name, reputation (number of stars received) and compliance score (completeness of documents).

CHANGE 6: Vendor should be able to accept and reject any booking made to any venue under their management.

**DI [5 marks]**

g. Implement the visual representation (infographic) feature for vendors from Assessment 1 with the following changes:

CHANGE 1: All the relevant details are stored in the MS SQL Server database. The API in the backend should handle all the database operations. All the form input validations must be handled on the React end, as well as being validated in the backend / REST API.

CHANGE 2: Vendor should see multiple charts showing:

1. All the hirers' tallies for each venue they manage in **bar chart** format.
2. All hirers' tallies for all the venues they manage combined as total in **stacked bar chart** format.
3. Most and least active hirers' names alongside their tallies (number of times they have made a booking in pie chart).

4. CHANGE 3: Venue utilization in **line chart** showing the ups-and-downs of utilization by hirers over time. Each vendor should be able to interact with the charts presented from a time perspective, e.g. zoom in and out to show stats for this week, this month, last month, and all time.

**ATTENTION: Postgraduate students (COSC2938)**

Refactor the existing React codebase to clearly separate data visualisation responsibilities from the structural and functional logic of components. *What architectural approach would you adopt to effectively separate visual representation from business logic, in a way that supports efficient future expansion of the codebase? Explain your reason via code comments in the appropriate file.*

*Proceed to next page for the HD section➡*

**HD [9 marks]**

**Note:** this section requires rigour, research, and effort. You will not find direct answers anywhere; you should do some reading on the Internet before devising your own implementations. **DO NOT COPY/PASTE** solutions from ChatGPT and/or CoPilot. **REMEMBER**, we will report plagiarism case to RMIT Student Misconduct Team who will organise a face-to-face interview to investigate a suspicious code.

**Your task is to create an Admin Dashboard with the following requirements:**

- **Requirement # 1: Create separate project(s) for this section i.e. admin react front (admin-frontend) & admin GraphQL (admin-backend). Admin Dashboard should not be linked to VV website.**
- **Requirement # 2: For the data fetching part (from the Cloud MS SQL), you must use GraphQL- i.e., use of REST API is not allowed.**
- **Requirement # 3: Admin must login with credentials (admin, admin) to access the dashboard.**

- h. (2 marks) An admin must be able to perform the following tasks:
- Assign vendor to venue(s). A vendor can have multiple venues, however a venue can only belong to one vendor. Sometimes a venue got sold to another vendor. This feature is to swap vendors.
  - Perform CRUD operation on all venues.
  - Select / deselect venues to appear on Featured Venues section on Hirers page.
- i. (2 marks) An admin should be able to generate the following reports:
- Top 3 most popular venues and their most popular day and time slot (e.g. Tuesday 9pm-11pm).
  - Top 3 most active applicants (successful bookings vs number of booking applications submitted).
- j. (2 marks) IMPLEMENT ONE OF THE FOLLOWING features:

Implement a real-time feature using **GraphQL subscriptions** that allows an admin to notify hirers and vendors when a specific venue becomes on sale with 45% discount. The venue name should appear red and bold with a warning in the frontend VV website both on the hirers and vendors dashboards simultaneously. You will receive **no marks if GraphQL subscriptions have not been used**.

OR

Add **6 contextual unit tests in the backend of the project (Node & Express)**. Explain the test via code comments in the relevant file.

- k. (3 marks) Deploy your website on Render and ensure each web service is live, up, running, and accessible via public URL:
- A web service for frontend
  - A web service for backend API connected with DB server
  - A web service for admin frontend
  - A web service for admin backend API connected with DB server via GraphQL

At the time of writing Render.com provides a Hobby Plan that costs \$0 USD per user / month plus compute costs. We expect that you should not be charged for free web service instances, with 512 MB RAM and 0.1 CPU which is very likely for a team of 2 and a marker to access your websites for few hours. Feel free to suspend and delete the services 2-3 weeks after you submitted your work.

Please include URL of each webservice in your README file.

## 2.8. Marking Guidelines

The marks allocated have been added to each of the tasks. **However, you will need to read the rubric for details.**